



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS
COMPONENTE: PEX0162 - ENGENHARIA DE SOFTWARE
DOCENTE: HULIANE MEDEIROS DA SILVA

Sistema de Gerenciamento Financeiro Pessoal

DISCENTES

CARLOS DANNIEL GONÇALVES DA SILVA
LEONARDO AUGUSTO SILVA DE SOUSA
PEDRO CARLOS LIMA PAIVA
SAMUEL IAGO DE FARIAS CABRAL

PAU DOS FERROS – RN

2026

1. Entendendo o Problema

- **Qual problema você observou?**

A dificuldade que as pessoas têm de controlar seus gastos, incertezas econômicas e falta de visibilidade para onde o dinheiro está indo no final do mês, incapacitando de poupar dinheiro ou planejar metas financeiras.

- **Quem são as pessoas envolvidas?**

Usuários finais, ou seja, todo o público que tenha interesse de ter uma organização financeira.

- **Como funciona hoje?**

A gestão financeira pessoal costuma ser descentralizada, fragmentada e muitas vezes analógica. Não existe uma educação formal de gestão financeira no ensino público.

- **Descreva como o processo acontece atualmente:**

O usuário ainda têm o costume de anotar os gastos em uma caderneta, que é uma forma muito antiga, mas que ainda podemos ver nos dias de hoje. Outros usam planilhas do Excel que exigem conhecimento básico para poderem utilizar, e uma parte dos usuários utilizam do extrato bancário para fazer cálculos de finanças.

- **Quais são as maiores dificuldades?**

Inconveniência dos métodos atuais, que exigem muita disciplina e tempo. Lembrar de despesas ao longo do dia e repassar para uma planilha, pode acontecer que alguma passe despercebida, e nos extratos bancários não categorizam despesas, dificulta a análise financeira ao final do mês.

- **Por que o software que vocês estão propondo ajudaria?**

Por ser um aplicativo web, o software oferecerá a conveniência de ser acessado de qualquer dispositivo com navegador e internet (computador, tablet, celular), facilitando o acompanhamento financeiro. Ele permitirá o registro de receitas e despesas de forma centralizada, categorização dos

gastos e esclarecerá para os usuários informações úteis e acessíveis, ajudando na sua saúde financeira.

2. Definição da Metodologia

- **Discutir e escolher qual modelo de processo de software será adotado:**
Modelo ágil (Possibilidade de mistura entre os papéis definidos do Scrum do que entregar a cada semana junto com o quadro Kanban definir tarefas pequenas e focar em uma de cada vez até que fique pronto).
- **Nome do modelo escolhido:**
Modelo híbrido entre o Kanban e Scrum (Scrumban) com visão de utilizar a prototipação para um MVP.
- **Justificativa da escolha.**
Pelo motivo da organização e prazos bem definidos de entrega por ambas. (Flexibilidade, demanda contínua, visualização do que é urgente , fluxo contínuo). Prototipação para cumprir requisitos e refinar potenciais.
- **Etapas previstas e entregas parciais.**
Levantamento de requisitos, modelagem, implementação, teste, entrega.
- **Papéis de cada integrante (ex.:gerente, analista, dev, testador, documentador).**
O Scrumban combina práticas do Scrum e do Kanban, priorizando a flexibilidade na organização do trabalho. Logo, os papéis na equipe não são rigidamente definidos. A seguir está uma previsão das responsabilidades de cada membro ao longo do projeto, podendo haver ajustes conforme a evolução das atividades e a colaboração entre os integrantes.

Product Owner (PO) & Backend Developer: Pedro Carlos Lima Paiva

Gestão: Responsável por definir as prioridades das funcionalidades no *Backlog*, detalhar os requisitos no quadro Kanban e garantir que o valor de negócio (controle financeiro) seja entregue.

Técnico: Lidera o desenvolvimento da lógica de negócio e APIs utilizando Python, Django e Django REST Framework.

Scrum Master & Frontend Developer: Leonardo Augusto Silva de Sousa

Gestão: Responsável por facilitar a comunicação do grupo, cobrar os prazos das tarefas no Kanban e remover impedimentos que travem o fluxo de trabalho.

Técnico: Lidera o desenvolvimento da interface do usuário e integração com a API utilizando Next.js e Tailwind CSS.

Core Developer & DevOps: Samuel Iago de Farias Cabral

Gestão: Responsável por definir o tamanho e a complexidade técnica das tarefas (estimativa) no quadro Kanban.

Técnico: Responsável pela modelagem e integridade do banco de dados PostgreSQL e pela configuração do ambiente isolado via Docker, garantindo a padronização entre os membros.

Analista de Qualidade (QA) & Documentador: Carlos Dannel Gonçalves da Silva

Gestão: Responsável por validar se as tarefas movidas para "Done" (Concluído) realmente atendem aos critérios de aceitação.

Técnico: Responsável pela elaboração dos relatórios técnicos e pela criação dos diagramas de arquitetura (Casos de Uso, Classes, DER) utilizando PlantUML, além de realizar os testes funcionais do sistema.

3. Montagem do Cronograma

- Montar o cronograma com marcos, prazos e responsáveis

CRONOGRAMA DE EXECUÇÃO DO PROJETO — PEX0162				
Engenharia de Software				
Metodologia: Scrum + Kanban · Equipe: Pedro Carlos, Carlos Danniel, Samuel e Leonardo · Prazo final: 15 de junho de 2026				
Período	Etapa	Atividades	Responsável(s)	Checkpoint / Marco
1. LEVANTAMENTO				
28 – 30 abr	Levantamento	- Definir o problema, usuários e contexto do sistema - Escolher e justificar a metodologia (Scrum + Kanban) - Definir stack tecnológica - Montar cronograma do projeto	Pedro Carlos Carlos Danniel Samuel Leonardo	▲ 30/abr — Entregar Plano de Stack Tecnológica
2. MODELAGEM				
01 – 06 mai	Modelagem	- Documento de Visão - Contexto, problema, escopo e stakeholders	Carlos Danniel Samuel	✓ Revisão interna do Documento de Visão
07 – 12 mai	Modelagem	- Documento de Requisitos (RF e RNF) - Priorização e critérios de aceitação	Carlos Danniel Samuel	✓ Revisão interna dos Requisitos
13 – 19 mai	Modelagem	- Diagramas: Caso de Uso, Classes e DER - Revisão geral de toda a documentação	Todos	▲ 19/mai — Entrega Unidade II
3. IMPLEMENTAÇÃO				
01 – 09 jun	Implementação	- Setup do repositório e ambiente de desenvolvimento - Estrutura do projeto (frontend + backend + banco) - Implementação dos RF 1, 2 e 3	Samuel Leonardo	✓ Demo interna: RF 1–3 funcionando
10 – 12 jun	Implementação	- Implementação dos RF 4, 5 e 6 - Implementação dos RNF 1 e 2 - Integração entre os módulos	Samuel Leonardo	✓ Demo interna: sistema completo e integrado
4. TESTES				
13 – 14 jun	Testes	- Testes funcionais de todos os RF - Verificação dos RNF - Correção de bugs identificados	Todos	✓ Todos os requisitos validados e aprovados

5. ENTREGA				
15 jun	Entrega	• Relatório final (metodologia, stack, resultados, lições aprendidas) • Preparação da apresentação e demo • Push final e organização do repositório • Envio do link ao professor	Todos	▲ 15/jun — Entrega final Unidade III ✓ Link do repositório enviado ao professor

4. Definição da Stack de Desenvolvimento

Cada equipe deve definir quais tecnologias utilizar no desenvolvimento, de forma justificada.

O que incluir:

1. Frontend: [Next.js](#).

Sendo um framework baseado em React, permite a criação de interfaces web dinâmicas e reativas, o que é essencial para um sistema financeiro.

2. Backend: Python3, Django e Django Rest Framework.

O Django traz painel de administração e sistemas de autenticação seguros de fábrica.

DRF torna a criação API REST que fará a comunicação com o [Next.js](#) rápida e eficiente.

3. Banco de Dados: PostgreSQL.

Banco de dados relacional seguro e confiável do mercado, fundamental para garantir a integridade dos dados, assegurando que nenhuma receita ou despesa seja salva com erros ou pela metade.

4. Ferramentas de versionamento: GitHub, Git.

Padrão global da indústria, permitindo que a equipe trabalhe no código simultaneamente, garantindo a integração contínua e o backup seguro do projeto.

5. Outras ferramentas: Docker, PlantUML.

O Docker isolará o banco de dados e as aplicações em contêineres, garantindo que todos da equipe tenham exatamente o mesmo ambiente.

O PlantUML agilizará a criação e atualização dos diagramas exigidos na documentação através do código.

6. Ambiente de desenvolvimento: Local com Docker.

Ideal para a fase atual de desenvolvimento e testes, garantindo custo zero de infraestrutura e velocidade na codificação antes da entrega final.

- **Possíveis limitações ou riscos.**

Apesar das vantagens, a stack adotada apresenta algumas limitações e riscos. O uso de múltiplas tecnologias aumenta a complexidade inicial do projeto, exigindo um conhecimento técnico da equipe;

O docker pode consumir mais recursos computacionais durante a execução, o que pode afetar máquinas com menor desempenho;

O PostgreSQL, apesar de ser um banco de dados robusto, pode ser um pouco mais difícil de configurar e administrar em comparação com opções mais simples;

A comunicação entre frontend e backend por meio das APIs REST exige cuidado com a segurança e com o tratamento adequado de dados trafegados.